

Benchmark Report — Task 6

Scalability evaluation of two storage strategies for the Taxi Trips dataset. Produced by `benchmark.py`; raw measurements are in `benchmark_results.json`.

1. Storage strategies evaluated

Both strategies store the **same 8,480,540 cleaned trips** with identical schema and identical row content. The only variable is the partition key, so any measured difference is attributable to physical layout alone.

	Strategy	Partition key	Partitions	Rationale
S1	Coarse temporal	year, month	8	The scheme the platform currently uses (Task 2)
S2	Fine temporal	year, month, day	96	One level finer, to test whether more, smaller partitions help or hurt

Both partition keys are temporal because `borough` **does not exist in the taxi trips dataset** — the trips carry only `pulocationid`, and `borough` is resolved from the `taxi_zones` lookup. Two of the three required queries are per-borough, so they join to `taxi_zones` at query time; that join is identical in both strategies and therefore cancels out of the comparison.

Partition counts exceed the 3 months of the study period because the trip data contains a handful of corrupt pickup timestamps dated 2002, 2008, 2009 and 2023, each of which creates its own partition.

2. Method

Metric	How it was measured
Ingestion time	Wall-clock duration of the Delta write
Storage size	Sum of parquet files referenced by the current Delta version , read from <code>_delta_log</code>
Generated files	Count of those same files
Query latency	2 warm-up runs discarded, then median of 5 timed runs; cold run reported separately

Three details matter for the numbers to mean anything:

- **Storage is not measured with `du`.** Delta's `mode("overwrite")` tombstones the previous version's files rather than deleting them, so a directory can be far larger than the live table. In this project `output_data/air_quality` measured 404 MB on disk while the live table was 2.5 MB — a 160× overstatement. Each strategy is also written to a fresh directory so no stale files can leak in.
- **Caches are dropped between runs.** `spark.catalog.clearCache()` is called and both DataFrames are re-loaded on every iteration, so each run genuinely re-reads the table.
- **Results are materialised with `.collect()`, not `.show()`,** which can short-circuit before the full aggregation completes.

Environment: PySpark 3.5.5, Delta Lake 3.1.0, OpenJDK 17.0.18 (arm64), Apple M2 Pro, 16 GB RAM, `local[4]`, `spark.sql.shuffle.partitions = 64`.

3. Results

Storage and ingestion

Metric	S1 year, month	S2 year, month, day	Difference
Ingestion time	12.40 s	11.84 s	within noise
Storage size	451.5 MB	448.9 MB	−0.6%
Generated files	14	278	20× more
Partitions	8	96	12× more
Average file size	33.0 MB	1.65 MB	20× smaller

Ingestion time is reported as a tie deliberately. Across two independent executions the ordering reversed (S2 was 14.15 s vs S1 12.86 s in the first, 11.84 s vs 12.40 s in the second), so the ~0.5 s gap is measurement noise, not a property of the strategies.

Query latency

Median of 5 timed runs, in seconds, with the observed range in brackets.

Query	S1 year, month	S2 year, month, day
Number of trips per borough	0.405 [0.357 – 0.528]	0.344 [0.334 – 0.360]
Average trip duration per day	0.362 [0.335 – 0.373]	0.306 [0.300 – 0.324]
Average fare per borough	0.364 [0.353 – 0.407]	0.356 [0.328 – 0.360]

Cold-run latencies were 1.130 / 0.683 / 0.498 s for S1 and 0.799 / 0.406 / 0.401 s for S2 — roughly 1.5–3× the warm figures, which is why they are excluded from the medians.

Query result validation

Timings are only meaningful if the queries are correct, so the outputs were checked and confirmed **identical across both strategies** for all three queries. Trips per borough sums to 8,480,540, exactly the cleaned trip count:

Pickup borough	Trips	Average fare
Manhattan	7,610,624	\$14.72
Queens	767,053	\$51.27
Brooklyn	58,126	\$27.82
Unknown	27,413	\$19.45
Bronx	15,530	\$33.49
N/A	1,509	\$64.61
EWR	181	\$74.74
Staten Island	104	\$28.18
Total	8,480,540	

The fare pattern is a useful sanity check in itself: Manhattan is lowest at \$14.72 because its trips are short intra-borough hops, while Queens (\$51.27) and EWR (\$74.74) are dominated by long airport runs from JFK, LaGuardia and Newark.

The per-day query returns 96 days rather than 91, for the same reason S2 has 96 partitions — the corrupt 2002/2008/2009/2023 pickup timestamps each contribute a day.

4. Discussion

Partition pruning never engages for this workload

The single most important observation is that **none of the three required queries contains a WHERE clause**. All three are full-table aggregations: they group by borough or by day, but they filter nothing. Partition pruning only triggers when a predicate restricts the partition column, so neither strategy can skip a single file. Both read all 8.48 million rows on every run.

This means the benchmark is *not* measuring the usual benefit of partitioning. It is measuring what partitioning costs when its benefit is unavailable — which is precisely the case Task 2 asks about under "when does partitioning become harmful."

Finer partitioning was slightly faster, not slower

S2 won two of the three queries by a margin that survives the noise. For *average trip duration per day* the ranges do not overlap at all (S1 0.335–0.373 s vs S2 0.300–0.324 s), a genuine ~15% improvement. *Trips per borough* shows a smaller but consistent edge, and *average fare per borough* is a tie within measurement error.

This is the opposite of the naive "small files are always bad" expectation, and the reason is read parallelism. S1 produces 14 files of 33 MB; S2 produces 278 files of 1.65 MB. With `local[4]`, 14 files yield 14 tasks across 4 cores — coarse and unevenly balanced — while 278 files yield far more, smaller tasks that keep all four cores busy and level out. At 1.65 MB per file, the per-file open/metadata overhead is still much cheaper than the parallelism it buys.

Where the small-files penalty actually begins

The penalty is real, but 1.65 MB files are not small enough to trigger it. This project already contains a table that is: `weather` is partitioned by `year`, `month`, `day` and produces **366 files averaging 13 KB** for only 8,783 rows and 4.8 MB of data. There, per-file overhead dominates completely — Spark opens 366 files to read less data than a single parquet file should hold.

The contrast gives a concrete threshold for our design: partitioning by day is appropriate for `trip_data` (~93,000 rows and 1.65 MB per day) and inappropriate for `weather` (24 rows and 13 KB per day). The deciding factor is not the partition count but the resulting file size, and the `weather` table should be repartitioned to `year`, `month` — or left unpartitioned entirely, given it is only 4.8 MB.

Storage size is essentially independent of the partition key

451.5 MB versus 448.9 MB is a 0.6% difference, with the *more* heavily partitioned table marginally smaller. Row-group compression works about equally well either way at this scale; day-partitioned files group temporally adjacent rows, which slightly improves column compression and offsets the extra per-file footer overhead. Partition choice is therefore not a lever for reducing storage here.

Limitations

- **No selective queries.** The three prescribed queries cannot demonstrate pruning. A query such as `WHERE year = 2024 AND month = 3` would let S1 skip 7 of 8 partitions and S2 skip 65 of 96, and would likely reverse the ranking by favouring whichever scheme matches the predicate's granularity. Any conclusion here applies to full-scan aggregation workloads only.
- **Single machine, single dataset size.** Everything runs `local[4]` on one laptop against 450 MB. On a real cluster, file count also drives driver-side task scheduling and metadata listing, which we cannot

observe at this scale.

- **OS page cache is not controlled.** Spark's cache is cleared between runs, but the operating system may still serve parquet files from RAM. Warm medians should be read as a best case; the cold-run column is the more pessimistic bound.
- **Sub-second queries.** All timings fall between 0.30 s and 0.53 s, where fixed overheads such as query planning are a meaningful fraction of the total. Differences below roughly 0.05 s should not be treated as significant.
- **S1 here is not byte-identical to the production table.** `benchmark.py` rewrites the trips from the stored Delta table, producing 14 files, whereas `data_ingestion.py` writes the same partitioning as 63 files because its input DataFrame arrives with a different shuffle layout. Both benchmark strategies were written from the same source DataFrame, so the S1-versus-S2 comparison remains fair.

5. Conclusion

For the prescribed workload the two strategies are close to equivalent: storage is within 0.6%, ingestion is a tie, and query latency differs by at most $\sim 15\%$ in S2's favour. The substantive difference is structural — S2 generates **20× more files** for no storage saving and no pruning benefit, because none of these queries filters on a partition column.

We therefore keep **S1 (year , month)** as the production layout for `trip_data`. It delivers comparable performance with 14 files instead of 278, which is the safer default as data volume grows and as the platform accumulates additional monthly loads. S2's small latency advantage comes from read parallelism that could be obtained more cheaply by tuning file sizes directly, without a $12\times$ increase in partition count.

The benchmark did surface one concrete defect in the current design: the `weather` table's `year , month , day` partitioning produces 366 files of 13 KB each and should be coarsened.